TP6 : le jeu du Chomp (2 heures)

Avant-propos

— Tous les TPs sont à réaliser **individuellement**.
— **Tous les TPs doivent être effectués sous Linux, en utilisant Emacs**.
— Le temps prévu en cours pour chaque TP est volontairement limité : il ne permet pas, en général, de terminer l'ensemble du sujet.
  - Il est donc **indispensable** de poursuivre le travail chez vous pour finaliser les exercices.
  - Terminer les TPs vous aidera à progresser efficacement, à consolider vos acquis et à être prêts pour les évaluations.
  - En cas de difficulté, n'hésitez pas à poser vos questions à la séance suivante.

La bibliothèque MLV est une bibliothèque multimédia simplifiée. Cette bibliothèque convient parfaitement aux étudiants qui débutent dans la programmation en C et qui souhaitent ajouter des effets graphiques et sonores dans leurs programmes.

La bibliothèque permet de : dessiner des figures, afficher du texte et des boîtes de dialogue, afficher des images, jouer de la musique, récupérer et traiter des évènements clavier et souris, récupérer des informations à l'aide d'une boîte de saisie de texte. Ces outils sont des interfaces simplifiées de la bibliothèque SDL (la suite sur : `http://www-igm.univ-mlv.fr/~boussica/mlv/api/French/html/index.html`).
NB : TP inspiré d'un TP proposé par Nicolas Borie : `www-igm.univ-mlv.fr/~borie/`

1 - Découvrir MLV

Visiter la page `http://www-igm.univ-mlv.fr/~boussica/mlv/api/French/html/index.html` et parcourir la documentation de la bibliothèque MLV.

S'assurer que la bibliothèque est bien installée en créant un programme `rectangle.c` qui affiche un rectangle rouge dans une fenêtre.

Créer un programme `rectangle_clic.c` qui affiche un rectangle rouge dans une fenêtre. Lorsque l'utilisateur clique sur le rectangle, il change de couleur.

2 - Le jeu du Chomp

L'objectif de cet exercice est d'implanter le jeu de Chomp, un jeu mathématique à 2 joueurs inventé par David Gales. Les règles sont les suivantes. Deux joueurs se disputent une tablette de chocolat de dimension $n \times m$ où n et m sont des entiers supérieurs à 1. Chaque joueur choisit alternativement un carré de chocolat, le mange et manque aussi tous les carrés qui se trouvent en bas à sa droite. La partie s'arrête lorsque l'un des joueurs mange le carré de chocolat en haut à gauche (i.e. en position (0,0)) : ce carré est empoisonné, le joueur qui mange ce carré perd donc la partie. Pour plus de détails sur le jeu, RDV sur la page Wikipédia du jeu de Chomp.
INDICATION : n'oubliez pas de tester au fur et à mesure vos fonctions.

3 - Définitions des types

Dans un fichier interface `types.h`, définir les types suivants :
— `tablette` : représente une tablette de chocolat de dimension `n`×`m`, autrement dit qui modélise le plateau de jeu. Utiliser un tableau d'entiers (short) de dimension `n`×`m`. Une case à 1 signifie que le carré de chocolat n'a pas encore été mangé, une case de à 0 indique qu'il a été mangé. Toute variable de type tablette connaît sa dimension, c'est-à-dire les entiers `n` et `m`.
— `joueur` : modélise les deux joueurs (utiliser un type énumération)
— `configuration` : représente une configuration du jeu de Chomp. Une configuration est déterminée par une tablette de chocolat et le joueur dont c'est le tour.
— `coup` : modélise un coup joué. Un coup est entièrement spécifié par les coordonnées `c` et `y` du carré de chocolat que le joueur souhaite manger.

4 - Manipulation des objets

Afin de manipuler les objets utiles au jeu, vous devez programmer les fonctions :

— `tablette creer_tablette(int n, int m)` : crée et retourne une variable de type `tablette` de dimension `n`×`m`. La tablette retournée possède tous ses carrés.

— `void manger(tablette * t, int x, int y)` : modifie la tablette pointée par `t` : on mange le carré en position `(x,y)` ainsi que tous ceux en dessous de lui et à sa droite.
RAPPEL : lorsque l'on effectue un passage par adresse d'une structure de données, l'accès aux champs s'effectue avec "une flèche" -> au lieu d'un point.

— `int est_legal(configuration *config, coup *cp)` : retourne 1 si le coup `cp` est légal dans la configuration de jeu `config`. Un coup est légal s'il ordonne de manger un carré de chocolat qui existe encore.

— `int est_finie(configuration *config, joueur *gagnant)` : retourne 1 si la partie `config` est terminée, 0 sinon. Dans le cas où la partie est terminée, la fonction affecte à la variable pointée par gagnant le joueur qui gagne la partie. La partie est terminée lorsque le carré de chocolat empoisonné vient d'être mangé. Dans ce cas-là, le joueur qui vient de jouer a perdu.

— `void jouer_coup(configuration *config, coup *cp)` : joue le coup `cp` dans la configuration `config`. Il ne faut pas oublier de modifier le champ qui contient le joueur dont c'est le tour de jouer.

5 - Assemblage du jeu

— Écrire une fonction `void afficher_configuration(configuration *config)` : affiche l'état de la partie en utilisant la bibliothèque MLV.

— Écrire une fonction coup `lire_coup(configuration *config)` : attend un clic de l'utilisateur sur la tablette dans la fenêtre graphique, calcule et retourne le coup spécifié par l'utilisateur. Si l'utilisateur ne clique pas sur la tablette, ou clique sur un carré déjà mangé, la fonction ne fait rien et attend de traiter le prochain clic.

— Utiliser les fonctions précédentes pour construire la fonction `main` qui permet au joueur de jouer au Chomp. Utiliser pour cela l'algorithme suivant :

1. récupérer la dimension de la tablette passée en argument au programme
2. initialiser la configuration `config`
3. tant que `config` ne représente pas une partie terminée :
   — afficher `config` sur la fenêtre graphique
   — lire un coup `c` sur la fenêtre graphique
   — jouer le coup `c` dans la `config`
   — afficher le numéro du joueur gagnant